

Access Genie API — Testing Reference

Every endpoint the API exposes, with headers, parameters, response shapes and a worked example of how to test each family.

Generated from the route table itself ([backend/src/routes/](#)), not from memory — if this disagrees with the code, the code is right and this file needs regenerating.

- **Companion documents:** [13-api-design.md](#) (design rationale), [12-database-design.md](#) (what the data looks like)
 - **Source of truth:** [backend/src/routes/index.ts](#)
-

1. Before you start

Base URL

Environment	Base URL
Local	http://localhost:4000/api/v1
Through a tunnel	<a href="https://<your-tunnel-domain>/api/v1">https://<your-tunnel-domain>/api/v1

The prefix [/api/v1](#) comes from `API_PREFIX` in [backend/.env](#). **A request to [/api/...](#) without the version returns 404** — the single most common mistake when first testing this API.

One endpoint sits *outside* the prefix:

```
GET /health      →  {"success":true,"data":  
{"status":"ok","database":"connected", ...}}
```

Use it as your liveness check. `database: "connected"` is the part that matters — the API answers on the port before Mongo is up.

Two processes must be running

```
npm run dev          # starts BOTH the API (:4000) and the web client (:5173)
```

`npm run dev:api` alone is enough for API testing.

Credentials

Read them from `backend/.env` — never from older docs, which are out of date:

Variable	Used for
<code>ADMIN_EMAIL</code> / <code>ADMIN_PASSWORD</code>	The real administrator account
<code>SEED_PASSWORD</code>	Demo personas — only exist if <code>npm run seed:demo</code> was run

Throughout this document `$EMAIL` and `$PASSWORD` stand for those values. Do not paste real credentials into shared files or Postman collections you export.

2. The response envelope

Every response — success or failure — uses one of these two shapes. There are no exceptions, which is what makes client-side handling uniform.

Success

```
{
  "success": true,
  "data": { "id": "AST-1042", "name": "Dell PowerEdge R740" }
}
```

Success, paginated (list endpoints)

```
{
  "success": true,
  "data": [ { "id": "AST-1042" }, { "id": "AST-1043" } ],
  "meta": {
```

```
"page": 1,
"limit": 25,
"total": 137,
"totalPages": 6,
"hasNext": true,
"hasPrev": false
}
}
```

Failure

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Request validation failed",
    "details": [ { "path": "category", "message": "Invalid option: expected one
of \"Compute\"|...\" } ]
  },
  "requestId": "3d039181-59b3-4a45-a0ce-da8b4f667688"
}
```

details appears only on validation failures, and each entry maps to one input field. **requestId** is echoed in the **x-request-id** response header and in the server log — quote it when reporting a bug.

Two guarantees worth testing for:

1. Mongo's **__id** is always serialised as **id**, and **__v** is always stripped. If you ever see **__id** in a response, that is a bug.
2. A response body **without success** did not come from this API — it came from a proxy, load balancer or tunnel in front of it. Treat it as "the API never received the request".

3. Headers

Request headers

Header	When	Value
Authorization	Every endpoint except the public auth ones	Bearer <accessToken>

Header	When	Value
Content-Type	Every POST / PATCH with a body	application/json
Cookie	POST /auth/refresh and /auth/logout only	Sent automatically by browsers
ngrok-skip-browser-warning	Only when testing through ngrok	1 — otherwise ngrok returns its HTML interstitial instead of JSON

Response headers

Header	Meaning
x-request-id	Correlates the response with the server log line
ratelimit / ratelimit-policy	Remaining budget in the current window (draft-7 format)
set-cookie	Only on /auth/login, /auth/refresh, /auth/mfa/verify

4. Authentication

The model

Two tokens, deliberately different:

Token	Lifetime	Where it lives	Sent how
Access token	15 minutes (JWT_ACCESS_TTL)	Memory — never localStorage	Authorization: Bearer ...
Refresh token	7 days (JWT_REFRESH_TTL)	httpOnly cookie, scoped to /api/v1/auth	Automatically, by the browser

The access token is short-lived on purpose, so a leaked one expires quickly. The refresh cookie is httpOnly (JavaScript cannot read it), sameSite-restricted (blunts CSRF) and path-scoped to /api/v1/auth — so it is not attached to ordinary API calls at all.

For API testing tools: you only need the access token. Get one from `/auth/login`, put it in a variable, and re-login when it expires. Chasing the refresh cookie in Postman is rarely worth the effort.

Note that the user record is **re-read from the database on every request**, not trusted from the token claims. So suspending an account or changing its role takes effect on the very next call — worth knowing when testing permissions.

Step 1 — Log in

```
curl -s -X POST http://localhost:4000/api/v1/auth/login \
-H 'Content-Type: application/json' \
-d '{"email":"'$EMAIL'", "password":"'$PASSWORD'"}'
```

Response 200:

```
{
  "success": true,
  "data": {
    "user": {
      "id": "U-001", "name": "Raj", "email": "...", "initials": "RA",
      "roleId": "super_admin", "title": "Super Admin",
      "homeScopeId": "ORG-1", "mfaEnabled": false, "status": "active",
      "lastLoginAt": "2026-08-05T12:20:58.532Z"
    },
    "role": { "id": "super_admin", "name": "Super Admin", "tier": "Platform",
    "modules": "*" },
    "modules": ["workspace", "assets", "tracking", "ai", "maintenance", "inventory",
    "operations", "analytics", "alerts", "compliance", "admin", "system"],
    "accessToken": "eyJhbGciOiJIUzI1NiIs... ",
    "expiresIn": 900
  }
}
```

`modules` is the list to check when an endpoint returns 403 — see §5.

If the account has MFA enabled, login returns a *challenge* instead of a session, and you must exchange it:

```
curl -s -X POST $BASE/auth/mfa/verify \
-H 'Content-Type: application/json' \
-d '{"challengeToken":"<from login>", "code":"123456"}'
```

Step 2 — Save the token

```
export BASE=http://localhost:4000/api/v1
export TOKEN=$(curl -s -X POST $BASE/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"'${EMAIL}',"password":"'${PASSWORD}'"}' \
  | python3 -c 'import sys,json;print(json.load(sys.stdin)["data"]
["accessToken"])' )

echo $TOKEN    # confirm it is not empty
```

Step 3 — Call anything

```
curl -s "$BASE/assets?limit=5" -H "Authorization: Bearer $TOKEN"
```

Auth endpoints

Method	Path	Auth	Body / Params	Notes
POST	/auth/login	Public	{email, password}	Rate limited. Sets refresh cookie
POST	/auth/refresh	Cookie	—	New access token from the cookie
POST	/auth/logout	Cookie	—	Revokes this session
GET	/auth/personas	Public	—	Seeded demo accounts (empty if not seeded)

Method	Path	Auth	Body / Params	Notes
POST	/auth/mfa/verify	Public	{challengeToken, code}	Completes an MFA login
GET	/auth/me	Bearer	—	Current user, role and modules
PATCH	/auth/me	Bearer	{name?, phone?, timezone?, title?}	Update own profile
POST	/auth/change-password	Bearer	{currentPassword, newPassword}	
POST	/auth/logout-all	Bearer	—	Revokes every session
GET	/auth/mfa	Bearer	—	MFA status
POST	/auth/mfa/setup	Bearer	—	Begin enrolment, returns secret
POST	/auth/mfa/enable	Bearer	{code}	Confirm enrolment
POST	/auth/mfa/disable	Bearer	{password}	
POST	/auth/mfa/recovery-codes	Bearer	{password}	Regenerate
GET	/auth/sessions	Bearer	—	Active sessions
POST	/auth/sessions/:id/revoke	Bearer	—	Kill one session

5. Permissions — why you get 403

Access is gated on **modules**, not roles. A role grants a set of modules; each endpoint requires one or more. Holding *any* of the required modules is enough.

The twelve modules:

```
workspace · assets · tracking · ai · maintenance · inventory
operations · analytics · alerts · compliance · admin · system
```

The seven roles:

```
super_admin · org_admin · facility_manager · maintenance_manager
technician · executive · security_officer
```

super_admin holds "*" — every module.

A 403 tells you exactly what was missing:

```
{ "success": false,
  "error": { "code": "FORBIDDEN",
             "message": "Your role does not grant access to: inventory" } }
```

Each endpoint family below lists its module gate. **Test this deliberately:** log in as a **technician** and confirm the admin endpoints refuse you. The client hides navigation as a courtesy — the server is what actually enforces it, and that is what your tests should prove.

6. Common parameters

Every list endpoint accepts

Param	Type	Default	Notes
page	integer ≥ 1	1	
limit	integer 1–200	25	Over 200 is rejected, not clamped
sort	string	varies	Field name; prefix - for descending (-createdAt)
q	string 1–120	—	Free-text search

CSV filters

Filters marked csv accept one value or several:

```
?status=Active  
?status=Active,Maintenance → matches either
```

Dates

ISO-8601, with or without a time: `2026-08-05` or `2026-08-05T12:00:00Z`.

Path IDs

Business identifiers, not Mongo ObjectIds: `AST-1042`, `WO-7`, `U-001`, `CLS-COMPUTER`, `FAC-HQ`. 1–64 characters.

7. Error codes

Code	HTTP	Means	What to do
<code>BAD_REQUEST</code>	400	Malformed request	Fix the request
<code>UNAUTHORIZED</code>	401	Missing/invalid token	Log in again
<code>TOKEN_EXPIRED</code>	401	Access token past its 15 min	Refresh or re-login
<code>FORBIDDEN</code>	403	Role lacks the module	Check §5
<code>NOT_FOUND</code>	404	No such record — or wrong URL	Check the <code>/api/v1</code> prefix
<code>CONFLICT</code>	409	Duplicate unique field (e.g. serial number)	Change the value
<code>VALIDATION_ERROR</code>	422	Body/query failed schema	Read <code>details[]</code>
<code>PAYLOAD_TOO_LARGE</code>	413	Upload over 5 MB	Shrink the file
<code>RATE_LIMITED</code>	429	Too many requests	Wait for the window to reset

Code	HTTP	Means	What to do
<code>INTERNAL_ERROR</code>	500	Server fault	Quote <code>requestId</code> when reporting

Rate limits

Scope	Window	Limit (dev)	Limit (prod)
General API	60 s	10,000	300
<code>/auth/login</code> , <code>/auth/refresh</code>	15 min	100	10

Auth limits count **failed** attempts only — a successful login does not consume budget. Watch the `ratelimit` response header while load testing.

8. Endpoint reference

Legend: **Gate** is the module required. All endpoints need `Authorization: Bearer` unless marked public.

8.1 Assets — gate `assets`

Method	Path	Purpose
GET	<code>/assets</code>	List, filtered and paginated
GET	<code>/assets/stats</code>	Counts by status, health, category
GET	<code>/assets/:id</code>	One asset
GET	<code>/assets/:id/profile</code>	Asset 360 — record + location + condition + history
POST	<code>/assets</code>	Create
POST	<code>/assets/bulk</code>	Update many at once
PATCH	<code>/assets/:id</code>	Partial update
DELETE	<code>/assets/:id</code>	Delete — also needs admin

Query params for **GET /assets**: `page`, `limit`, `sort`, `q`, plus `status` csv, `category` csv, `health` csv, `criticality` csv, `trackingTech` csv, `facility`, `tracked` (boolean).

```
# Filtered list
curl -s "$BASE/assets?
status=Active,Maintenance&category=Compute&limit=10&sort=-healthScore" \
-H "Authorization: Bearer $TOKEN"

# Create — minimum viable body
curl -s -X POST "$BASE/assets" \
-H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
-d '{
  "name": "Test Server 01",
  "category": "Compute",
  "serialNumber": "SN-TEST-0001",
  "location": { "id": "FAC-HQ", "name": "Bengaluru HQ" }
}'
```

Field notes for create:

- `id` is optional — omit it and the server mints the next `AST-...`. If you supply one it must match `AST-\d+`.
- `category` must be one of the twelve in §11.
- `custodian` and `purchaseDate` are optional here by design ("commit early, enrich forever") — an unclaimed asset is stored as `Unassigned` and dated from its registration.
- `serialNumber` is unique — reusing one returns **409 CONFLICT**. Good test case.
- `healthStatus` is derived server-side; sending it is ignored.

8.2 Work orders — gate maintenance

Method	Path	Purpose
GET	<code>/work-orders</code>	List
GET	<code>/work-orders/stats</code>	Open/overdue/closed counts, MTTR, MTBF
GET	<code>/work-orders/:id</code>	One
POST	<code>/work-orders</code>	Create
PATCH	<code>/work-orders/:id</code>	Update
DELETE	<code>/work-orders/:id</code>	Delete

Method	Path	Purpose
POST	<code>/work-orders/:id/status</code>	Transition status
POST	<code>/work-orders/:id/comments</code>	Add a comment
POST	<code>/work-orders/:id/labor</code>	Log labour hours
POST	<code>/work-orders/:id/checklist</code>	Complete checklist items

Query params: the common four, plus `status` csv, `priority` csv, `type` csv, `assetId`, `assignedTo`, `overdue` (bool), `aiGenerated` (bool).

```
curl -s "$BASE/work-orders?overdue=true&priority=Critical,High" \
-H "Authorization: Bearer $TOKEN"

curl -s -X POST "$BASE/work-orders" \
-H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
-d '{"title":"Replace failing drive","assetId":"AST-16","priority":"High","type":"Corrective"}'
```

8.3 Alerts — gate `alerts` or `compliance`

Method	Path	Purpose
GET	<code>/alerts</code>	List — <code>status</code> , <code>severity</code> , <code>type</code> , <code>assetId</code>
GET	<code>/alerts/stats</code>	Counts by severity and status
GET	<code>/alerts/:id</code>	One
POST	<code>/alerts</code>	Raise one manually
POST	<code>/alerts/:id/acknowledge</code>	Acknowledge
POST	<code>/alerts/:id/escalate</code>	Escalate
POST	<code>/alerts/:id/resolve</code>	Resolve
POST	<code>/alerts/:id/assign</code>	Assign to a user
POST	<code>/alerts/bulk/acknowledge</code>	Acknowledge many

8.4 Dashboard, dataset and scope

Method	Path	Gate	Purpose
GET	/dashboard/summary	workspace	Every KPI, chart and list on the dashboard
GET	/dataset	—	The aggregate hydration payload the client boots from
GET	/scope/tree	—	Group ► Org ► Region ► Facility ► ... as a tree
GET	/scope	—	The same nodes, flat
POST	/scope	admin	Create a node
PATCH	/scope/:id	admin	Rename / re-parent
DELETE	/scope/:id	admin	Remove

GET /dashboard/summary — the most parameterised endpoint in the API:

Param	Values	Default
scope	Any scope node id (ORG-1, FAC-HQ)	All locations
period	7d 30d 90d 12m	30d
from / to	ISO dates — a custom window	—
department	Free text, matched against the asset's department	All
category	One of the twelve categories	All

Unrecognised values **fall back rather than fail** — a stale bookmark should still render a dashboard. So `?period=banana` returns 200 with 30 days, not 422. Test for that behaviour, not against it.

```
curl -s "$BASE/dashboard/summary?scope=ORG-1&period=90d&category=Compute" \
-H "Authorization: Bearer $TOKEN"

curl -s "$BASE/dashboard/summary?from=2026-07-01&to=2026-07-31" \
-H "Authorization: Bearer $TOKEN"
```

The response has four top-level blocks: `meta` (what was applied), `kpis`, `charts`, `lists`. Which keys appear depends on the caller's modules — a technician's response genuinely omits the finance keys rather than nulling them.

8.5 Tracking — gate tracking

Method	Path	Purpose
POST	/tracking/observations	Record one tag read
POST	/tracking/observations/batch	Bulk ingest — the reader/gateway path
GET	/tracking/observable-zones	Zones that can be observed
GET	/tracking/workspace	The whole tracking screen payload
GET	/tracking/live	Current presence
GET	/tracking/alerts/count	Open tracking alerts
GET	/tracking/movement/:id	One asset's movement history
GET/POST/DELETE	/tracking/sensors, /tracking/sensors/:id	Sensors
GET/POST/PATCH/DELETE	/tracking/gateways, /tracking/gateways/:id	Gateways (write needs admin)
GET/POST/PATCH/DELETE	/tracking/geofences, /tracking/geofences/:id	Geofences
POST	/tracking/alerts/bulk/transition	Move many alerts
POST	/tracking/alerts/:id/transition	Move one
POST	/tracking/incidents	Open an incident
POST	/tracking/incidents/:id/state	Change incident state

Method	Path	Purpose
POST	/tracking/automation-rules/:id/toggle	Enable/disable a rule
POST	/tracking/devices	Provision a device
POST	/tracking/devices/bulk	Bulk device update
POST	/tracking/firmware-campaigns/:id/state	Start/stop a rollout
POST/PATCH	/tracking/movements, /tracking/movements/:id	Movements
POST/PATCH	/tracking/audits, /tracking/audits/:id	Physical audits

8.6 Users and roles — gate **admin**

Method	Path	Extra requirement
GET	/users	filters <code>roleId</code> , <code>status</code>
GET	/users/:id	
POST	/users	role must be <code>super_admin</code> or <code>org_admin</code>
PATCH	/users/:id	
DELETE	/users/:id	role must be <code>super_admin</code>
GET	/users/roles	
PATCH	/users/roles/:id	Edit a role's module grants
POST	/users/roles/:id/reset	Restore shipped defaults

Editing role grants is the highest-leverage test in the suite: change a role's modules, then confirm a user in that role immediately gains or loses access — no re-login needed.

8.7 Asset classes and registry — gate **assets**

Method	Path	Notes
GET	/asset-classes	
GET	/asset-classes/:id	
POST	/asset-classes	{name, icon, category}
PATCH	/asset-classes/:id	
DELETE	/asset-classes/:id	needs admin
GET	/asset-groups, /asset-groups/:id	
GET	/asset-documents	
POST	/asset-documents	Upload — 5 MB limit
GET	/asset-documents/:id/download	Returns the file, not JSON
DELETE	/asset-documents/:id	
GET	/movement-trails, /movement-trails/:id	gate tracking

```
# A good validation test – 'Furniture' is not a category
curl -s -X POST "$BASE/asset-classes" \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"name":"Test Class","icon":"📦","category":"Furniture"}'
# → 422 VALIDATION_ERROR listing all twelve valid options
```

8.8 Maintenance programme — gate maintenance

Method	Path
GET/POST	/pm-schedules
GET/PATCH/DELETE	/pm-schedules/:id
POST	/pm-schedules/run-automation
GET	/inspections, /inspections/:id
POST/PATCH/DELETE	/inspections, /inspections/:id (gate compliance)
GET	/checklist-templates

Method	Path
POST/PATCH/DELETE	/checklist-templates, /checklist-templates/:id
GET	/field/queue
GET	/field/queue/all
POST	/field/scan/:id

8.9 Compliance — gate compliance

Method	Path
GET	/certifications
POST/PATCH/DELETE	/certifications, /certifications/:id
GET	/cycle-counts (gate operations/inventory)
POST/PATCH/DELETE	/cycle-counts, /cycle-counts/:id
GET	/audit
GET	/custody
POST	/custody
GET	/compliance-frameworks
GET/POST/PATCH/DELETE	/retention-policies

8.10 Inventory and procurement — gate inventory

Method	Path
GET	/inventory/parts
POST/PATCH/DELETE	/inventory/parts, /inventory/parts/:id
POST	/inventory/parts/:id/adjust — stock adjustment
GET	/inventory/parts/:sku/movements
GET/POST/PATCH/DELETE	/inventory/warehouses
GET/POST/PATCH/DELETE	/inventory/suppliers

Method	Path
GET/POST/PATCH	/inventory/purchase-orders
POST	/inventory/purchase-orders/:id/receive
GET	/inventory/reorder — SKUs at or below reorder point
POST	/inventory/reorder/draft — draft POs from that list

8.11 Operations — gate **operations** or **assets**

Method	Path
GET/POST	/operations/transfers
POST	/operations/transfers/:id/status
GET/POST	/operations/reservations
POST	/operations/reservations/:id/cancel

8.12 AI and insights — gate **ai**

Method	Path
GET	/insights , /insights/stats , /insights/:id
POST	/insights/:id/action , /insights/:id/dismiss
POST	/intelligence/recompute — re-derive health, risk, utilization, book value
GET	/intelligence/explain/:id — why an asset scores as it does
GET/POST/PATCH/DELETE	/ai/models , /ai/models/:id
GET	/ai/forecasts , /ai/forecasts/:id
GET	/ai/anomalies

POST [/intelligence/recompute](#) is a **write across the whole estate** — it rewrites derived columns on every asset. Safe (it is idempotent and the scheduler runs it every 10

minutes anyway) but do not benchmark with it.

8.13 Labels and printing — gate **assets**

Method	Path
GET/POST	/labels/templates
GET/PATCH/DELETE	/labels/templates/:id
GET	/labels/devices · POST needs admin
GET/POST	/labels/jobs
POST	/labels/jobs/:id/cancel , /labels/jobs/:id/retry

8.14 Reporting — gate **analytics**

Method	Path
GET/POST	/reports
GET/PATCH/DELETE	/reports/:id
POST	/reports/:id/run
GET	/exports/:id/download — returns a file
GET/POST/PATCH/DELETE	/report-subscriptions
GET	/report-packs

8.15 Platform administration — gate **admin / system**

Method	Path	Writable
GET	/org-settings	PATCH
GET/POST/PATCH/DELETE	/teams	✓
GET	/api-keys	POST/PATCH/DELETE (secret shown once)
GET/POST/PATCH/DELETE	/webhooks	✓

Method	Path	Writable
GET	/backups	POST /backups, POST /backups/:id/restore
GET	/invoices	read-only
GET/POST/PATCH/DELETE	/exports	✓
GET/POST/PATCH/DELETE	/support-tickets	✓
GET/POST/PATCH/DELETE	/escalation-policies	✓
GET	/on-call	read-only
GET/POST/PATCH/DELETE	/integrations	✓
GET/POST/PATCH/DELETE	/approval-workflows	✓
POST/DELETE	/passkeys, /passkeys/:id	
GET/POST/PATCH/DELETE	/alert-rules + /alert-rules/:id/toggle	gate alerts/compliance

8.16 Personal and notifications

Method	Path	Purpose
GET	/me/preferences	Theme, active scope, saved views, dashboard layout
PATCH	/me/preferences	Update any subset
POST	/me/views	Save a filter view
PATCH	/me/views/:id	Rename / redefine
DELETE	/me/views/:id	
GET	/notifications	
POST	/notifications/:id/read	
POST	/notifications/read-all	

9. How to test

9.1 curl — the four shapes

```
export BASE=http://localhost:4000/api/v1
export TOKEN="..." # from §4

# READ one
curl -s "$BASE/assets/AST-16" -H "Authorization: Bearer $TOKEN"

# READ many, filtered
curl -s "$BASE/assets?status=Active&limit=5&sort=-healthScore" \
  -H "Authorization: Bearer $TOKEN"

# CREATE
curl -s -X POST "$BASE/work-orders" \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"title":"Test work order","assetId":"AST-16","priority":"Low"}'

# UPDATE (PATCH — send only what changes)
curl -s -X PATCH "$BASE/assets/AST-16" \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"status":"Maintenance"}'
```

Add `-i` to see status and headers, or `-w '\n%{http_code}\n'` for just the code.

9.2 Postman / Insomnia

1. Create an environment with `base_url = http://localhost:4000/api/v1`, plus empty `token`.
2. In the **login** request, add a Test script so every later request is authenticated automatically:

```
// Tests tab of POST {{base_url}}/auth/login
const body = pm.response.json();
pm.environment.set("token", body.data.accessToken);
pm.test("login ok", () => pm.expect(body.success).to.be.true);
```

3. On the **collection** (not each request), set Authorization → Bearer Token → `{{token}}`. Every request inherits it.
4. Re-run login when you start getting `TOKEN_EXPIRED` — 15 minutes.

Useful collection-level test, since the envelope is universal:

```
pm.test("uses the standard envelope", () => {
  pm.expect(pm.response.json()).to.have.property("success");
});
pm.test("never leaks _id", () => {
  pm.expect(pm.response.text()).to.not.include('"_id"');
});
```

9.3 A smoke test you can run now

Save as `smoke.sh`, `chmod +x smoke.sh`, run it. It checks the whole happy path and cleans up after itself.

```
#!/usr/bin/env bash
set -euo pipefail
BASE=${BASE:-http://localhost:4000/api/v1}
EMAIL=${EMAIL:?set EMAIL}
PASSWORD=${PASSWORD:?set PASSWORD}
pass=0; fail=0

check () { # check <name> <expected-code> <actual-code>
  if [ "$2" = "$3" ]; then echo " ✓ $1"; pass=$((pass+1));
  else echo " ✗ $1 — expected $2, got $3"; fail=$((fail+1)); fi
}

code () { curl -s -o /tmp/ag_body -w '%{http_code}' "$@"; }

echo "— liveness"
check "health" 200 "$(code "${BASE%/api/v1}/health")"

echo "— auth"
check "login" 200 "$(code -X POST "$BASE/auth/login" -H 'Content-Type: application/json' -d '{"email":"'${EMAIL}',"password":"'${PASSWORD}'"}")"
TOKEN=$(python3 -c 'import json;print(json.load(open("/tmp/ag_body"))["data"]["accessToken"])')
AUTH="Authorization: Bearer $TOKEN"
check "wrong password" 401 "$(code -X POST "$BASE/auth/login" -H 'Content-Type: application/json' -d '{"email":"'${EMAIL}',"password":"'nope'"}")"
check "no token" 401 "$(code "$BASE/assets")"
check "garbage token" 401 "$(code "$BASE/assets" -H 'Authorization: Bearer abc')"
check "me" 200 "$(code "$BASE/auth/me" -H "$AUTH")"

echo "— reads"
for p in assets work-orders alerts dashboard/summary dataset scope/tree me/preferences notifications; do
  check "GET /$p" 200 "$(code "$BASE/$p" -H "$AUTH")"
done

echo "— filters"
check "csv filter" 200 "$(code "$BASE/assets?status=Active,Maintenance" -H "$AUTH")"
check "pagination" 200 "$(code "$BASE/assets?page=1&limit=5" -H "$AUTH")"
check "limit over max" 422 "$(code "$BASE/assets?limit=999" -H "$AUTH")"
```

```

check "bad period falls back" 200 "$(code "$BASE/dashboard/summary?
period=banana" -H "$AUTH")"

echo "— errors"
check "unknown id" 404 "$(code "$BASE/assets/AST-does-not-exist" -H
"$AUTH")"
check "missing prefix" 404 "$(code "${BASE%/v1}/assets" -H "$AUTH")"
check "bad category" 422 "$(code -X POST "$BASE/asset-classes" -H "$AUTH" -
H 'Content-Type: application/json' -d
'{"name":"X","icon":"📦","category":"Furniture"}')")

echo "— write, then clean up"
check "create asset" 201 "$(code -X POST "$BASE/assets" -H "$AUTH" -H
'Content-Type: application/json' \
-d '{"name":"Smoke Test Asset","category":"Compute","serialNumber":"SN-SMOKE-
001","location":{"id":"FAC-HQ","name":"HQ"}}')")
NEW=$(python3 -c 'import json;print(json.load(open("/tmp/ag_body"))["data"]
["id"])' 2>/dev/null || echo "")
if [ -n "$NEW" ]; then
    check "duplicate serial" 409 "$(code -X POST "$BASE/assets" -H "$AUTH" -H
'Content-Type: application/json' \
-d '{"name":"Dup","category":"Compute","serialNumber":"SN-SMOKE-
001","location":{"id":"FAC-HQ","name":"HQ"}}')")
    check "patch" 200 "$(code -X PATCH "$BASE/assets/$NEW" -H "$AUTH"
-H 'Content-Type: application/json' -d '{"status":"Maintenance"}')")
    check "delete" 204 "$(code -X DELETE "$BASE/assets/$NEW" -H
"$AUTH")"
fi

echo
echo "passed: $pass   failed: $fail"
[ "$fail" -eq 0 ]

```

```

BASE=http://localhost:4000/api/v1 EMAIL='...' PASSWORD='...' ./smoke.sh

```

9.4 What to cover beyond the happy path

Category	Test	Expect
Auth	No Authorization header	401 UNAUTHORIZED
Auth	Expired token (wait 15 min)	401 TOKEN_EXPIRED
Auth	Token from a deleted/suspended user	401 / 403 immediately, not on expiry
Permission	Technician calls /users	403 naming the missing module
Permission	Widen a role's grants, retry	200 on the very next call
Validation	Field over max length	422 with details[].path

Category	Test	Expect
Validation	Enum value not in the list	422 listing valid options
Validation	Empty string in an optional field	Accepted, treated as absent
Uniqueness	Duplicate serial number	409 CONFLICT
Pagination	limit=999	422 (max is 200)
Pagination	page past the end	200, empty data , correct meta
Routing	Missing /v1	404
Not found	Valid-shaped id that does not exist	404, not 500
Rate limit	100+ failed logins in 15 min	429 RATE_LIMITED
Scope	?scope= a facility with no assets	200 with zeroes, not an error

9.5 Testing through a tunnel

Add one header, or ngrok returns its HTML warning page instead of your JSON:

```
curl -s "$BASE/assets" -H "Authorization: Bearer $TOKEN" \
  -H 'ngrok-skip-browser-warning: 1'
```

In Postman, add it at collection level.

10. Gotchas that cost time

1. **/api/v1, not /api.** A missing version segment gives 404, which reads like a broken endpoint.
2. **A non-envelope error body means the API never saw the request** — a proxy or tunnel answered instead. Check the API is actually running on :4000.
3. **PATCH is partial.** Send only the fields you are changing; a PATCH is not a replace.
4. **Access tokens last 15 minutes.** Long test runs need a re-login step.
5. **Role changes are immediate** — the user is re-read on every request, so there is no cache to wait out.
6. **limit caps at 200** and rejects above it rather than clamping.
7. **The dashboard tolerates bad input by design.** Unknown **period** or **category** values fall back instead of 422-ing. Do not write a test asserting the opposite.

8. **The response shape depends on the caller's modules.** Missing keys in `/dashboard/summary` may be correct, not a bug — check the role first.
 9. **Downloads are not JSON.** `/asset-documents/:id/download` and `/exports/:id/download` return file bytes.
 10. **You are testing against the live Atlas database** unless `MONGODB_URI` says otherwise. Every asset you create is real. Clean up, or point at a scratch database first.
-

11. Reference values

Asset categories (12) `Compute` `Storage` `Network` `Endpoints` `Mobile` `Peripherals` `Accessories` `Audio` `Visual` `Security` `Software` `Infrastructure` `Sensors`

Asset statuses `Active` `Maintenance` `Missing` `End_Of_Life` `Staging`

Asset health `Good` `Warning` `Critical`

Criticality `Low` `Medium` `High` `Critical`

Tracking tech `RFID` `BLE` `GPS` `QR` `UWB` `LoRaWAN`

Dashboard periods `7d` `30d` `90d` `12m`

Scope levels `group` `org` `region` `facility` `building` `floor` `zone`

Modules `workspace` `assets` `tracking` `ai` `maintenance` `inventory` `operations` `analytics` `alerts` `compliance` `admin` `system`

Roles `super_admin` `org_admin` `facility_manager` `maintenance_manager` `technician` `executive` `security_officer`

The authoritative lists live in `shared/src/domain.ts` and `shared/src/platform.ts`. If you add a value there, update this section.